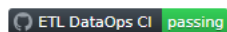


TP DataOps ETL

pandas + PostgreSQL + pytest + GitHub Actions

DRISS Ryma - M2 Data Engineer
Encadré par Dr Abdelhak Touiti



Introduction

Ce TP a pour objectif de réaliser un pipeline ETL DataOps complet utilisant :

- Python
- pandas
- PostgreSQL
- Docker
- pytest
- GitHub Actions

Le pipeline suit l'architecture :

<code>Extract -> Transform -> Load</code>

Le projet contient également :

- des tests unitaires
- des tests d'intégration
- l'automatisation CI/CD avec GitHub Actions
- un bonus de couverture de tests supérieur à 80%

Structure du projet

Le projet a été organisé selon l'arborescence suivante :

```
NOM_PRENOM/  
  
src/  
    extract.py  
    transform.py  
    load.py  
    run.py  
    __init__.py  
  
data/  
    ventes.csv  
  
tests/  
    conftest.py  
    test_transforms.py  
    test_load.py  
  
.github/workflows/  
    ci.yml  
  
docker-compose.yml  
requirements.txt  
README.md  
.gitignore
```

Création et lancement de PostgreSQL avec Docker

Création du fichier docker-compose.yml

Le fichier `docker-compose.yml` a été créé afin de lancer PostgreSQL comme entrepôt de données.

Lancement du conteneur Docker

```
docker compose up -d  
docker ps
```

```
ryma@DRISSRYMAHP:/mnt/c/Users/DRISS/Desktop/DRISS_Ryma_Devops/NOM_PRENOM$ docker compose up -d  
[+] up 12/17  
[+] up 12/17. [#####] 16.79MB / 161.2M  
[+] up 12/17. [#####] 19.94MB / 161.2M  
[+] up 12/17. [#####] 22.04MB / 161.2M  
[+] up 12/17. [#####] 24.13MB / 161.2M  
[+] up 12/17. [#####] 28.33MB / 161.2M  
[+] up 12/17. [#####] 30.54MB / 161.2M  
[+] up 12/17. [#####] 33.57MB / 161.2M
```

```

✓ Image postgres:15 Pulled 13.6s
✓ Network nom_prenom_default Created 0.1s
✓ Volume nom_prenom_pgdata Created 0.0s
:: Container etl_warehouse Starting 1.5s

```

```

ryma@DRISSRYMAHP:/mnt/c/Users/DRISS/Desktop/DRISS_Ryma_Devops/NOM_PRENOM$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS
1211bed4aab2   postgres:15                        "docker-entrypoint.s..." 41 seconds ago   Up 40 seconds
cp, [::]:5433->5432/tcp
etl_warehouse

```

Vérification de PostgreSQL

Une connexion à PostgreSQL a été effectuée depuis le conteneur Docker.

```
docker exec -it etl_warehouse psql -U etl_user -d warehouse
```

Les bases suivantes ont été affichées :

- postgres
- template0
- template1
- warehouse

List of databases							
Name	Owner	Encoding	Collate	Ctype	ICU Locale	Locale Provider	Access privileges
postgres	etl_user	UTF8	en_US.utf8	en_US.utf8		libc	
template0	etl_user	UTF8	en_US.utf8	en_US.utf8		libc	=c/etl_user +
template1	etl_user	UTF8	en_US.utf8	en_US.utf8		libc	=c/etl_user +
warehouse	etl_user	UTF8	en_US.utf8	en_US.utf8		libc	etl_user=CTc/etl_user
(4 rows)							

Création du fichier source CSV

Le fichier `data/ventes.csv` contient les données de ventes fournies dans le sujet.
Le dataset contient :

- un email vide
- un montant négatif

Ces données invalides doivent être supprimées durant la phase de transformation.

Implémentation de `extract.py`

Le fichier `extract.py` contient la fonction :

```
extract(csv_path)
```

Cette fonction :

- lit le fichier CSV
- retourne un DataFrame pandas brut

Exemple d'utilisation :

```
from src.extract import extract

df = extract("data/ventes.csv")
print(df.head())
```

Résultat obtenu

- 10 lignes ont été lues depuis le fichier CSV

```
>>> from src.extract import extract

df = extract("data/ventes.csv")

print(df.head())
>>>
>>> df = extract("data/ventes.csv")
[Extract] 10 lignes lues depuis data/ventes.csv
>>>
>>> print(df.head())
   id  client_email  ...  montant  categorie
0  1  alice@mail.com  ...   120.50  Electronique
1  2   bob@mail.com  ...    45.00    Vetements
2  3  alice@mail.com  ...   320.00  Electronique
3  4  carol@mail.com  ...    89.90      Maison
4  5   bob@mail.com  ...   210.00  Electronique
```

Implémentation de transform.py

Le fichier `transform.py` contient toutes les transformations pandas.

Fonction `clean(df)`

La fonction `clean(df)` réalise :

- conversion des types
- nettoyage des emails

- suppression des valeurs invalides
- suppression des montants négatifs
- suppression des dates invalides

Les emails sont convertis en minuscules.

Les lignes invalides sont supprimées.

Résultat obtenu

Après nettoyage :

- 8 lignes valides restent
- 2 lignes invalides sont supprimées

```
>>> from src.extract import extract
from src.transform import clean, aggregate_by_month

df = extract("data/ventes.csv")

df_clean = clean(df)

print(df_clean)

mart = aggregate_by_month(df_clean)

print(mart)>>> from src.transform import clean, aggregate_by_month
>>>
>>> df = extract("data/ventes.csv")
[Extract] 10 lignes lues depuis data/ventes.csv
```

```

'''
>>> df_clean = clean(df)
>>>
>>> print(df_clean)
   id  client_email date_vente  montant  categorie
0   1  alice@mail.com 2024-01-05   120.5  Electronique
1   2   bob@mail.com 2024-01-12    45.0    Vetements
2   3  alice@mail.com 2024-01-18   320.0  Electronique
3   4  carol@mail.com 2024-02-03    89.9      Maison
4   5   bob@mail.com 2024-02-14   210.0  Electronique
5   6  dave@mail.com 2024-02-20    15.5    Vetements
6   8  carol@mail.com 2024-03-08   430.0  Electronique
7   9  alice@mail.com 2024-03-15    75.0      Maison
>>>
>>> mart = aggregate_by_month(df_clean)
>>>
>>> print(mart)
   mois  chiffre_affaires  nb_transactions
0 2024-01             485.5                3
1 2024-02             315.4                3
2 2024-03             505.0                2

```

Fonction aggregate__by__month(df)

Cette fonction réalise une agrégation mensuelle du chiffre d'affaires.
Elle calcule :

- le chiffre d'affaires mensuel
- le nombre de transactions par mois

Résultat obtenu

Mois	Chiffre d'affaires	Transactions
2024-01	485.5	3
2024-02	315.4	3
2024-03	505.0	2

```
ryma@DRISSRYMAHP:/mnt/c/Users/DRISS/Desktop/DRISS_Ryma_Devops/NOM_PRENOM$ docker exec -it
etl_warehouse psql -U etl_user -d warehouse
psql (15.18 (Debian 15.18-1.pgdg13+1))
Type "help" for help.
```

```
warehouse=# SELECT * FROM ventes_propres;
```

```
SELECT * FROM ca_par_mois;
```

```
SELECT * FROM run_log;
  mois | chiffre_affaires | nb_transactions
-----+-----+-----
2024-01 |          485.5 |              3
2024-02 |          315.4 |              3
2024-03 |           505 |              2
(3 rows)
```

```
 id | run_at | etape | statut | nb_lignes | message
-----+-----+-----+-----+-----+-----
  1 | 2026-05-18 16:47:15.523346 | extract | success |      10 |
  2 | 2026-05-18 16:47:15.58185 | transform | success |       8 |
  3 | 2026-05-18 16:47:15.81518 | load | success |      11 |
(3 rows)
```

Bonus : aggregate_by_category(df)

Cette fonction calcule :

- le chiffre d'affaires par catégorie
- le nombre de transactions par catégorie

Implémentation de load.py

Le fichier `load.py` contient les fonctions de chargement PostgreSQL.
Les tables suivantes sont créées automatiquement :

- `ventes_propres`
- `ca_par_mois`
- `run_log`

Les DataFrames pandas sont chargés dans PostgreSQL avec :

```
df.to_sql()
```

La table `run_log` enregistre :

- l'étape exécutée
- le statut
- le nombre de lignes
- les erreurs éventuelles

id	client_email	date_vente	montant	categorie
1	alice@mail.com	2024-01-05 00:00:00	120.5	Electronique
2	bob@mail.com	2024-01-12 00:00:00	45	Vetements
3	alice@mail.com	2024-01-18 00:00:00	320	Electronique
4	carol@mail.com	2024-02-03 00:00:00	89.9	Maison
5	bob@mail.com	2024-02-14 00:00:00	210	Electronique
6	dave@mail.com	2024-02-20 00:00:00	15.5	Vetements
8	carol@mail.com	2024-03-08 00:00:00	430	Electronique
9	alice@mail.com	2024-03-15 00:00:00	75	Maison

Implémentation de run.py

Le fichier `run.py` orchestre tout le pipeline ETL.

Ordre d'exécution :

```
Extract -> Transform -> Load
```

Étapes exécutées

Étape 1 : Extract

Lecture du CSV.

Étape 2 : Transform

Nettoyage pandas et agrégation.

Étape 3 : Load

Chargement des données dans PostgreSQL.

Résultat obtenu

Le pipeline s'exécute avec succès.

```
ryma@DRISSRYMAHP:/mnt/c/Users/DRISS/Desktop/DRISS_Ryma_Devops/NOM_PRENOM$ python3 src/run.py
=== Pipeline ETL DataOps ===
```

```
[1/3] Extract ...
[Extract] 10 lignes lues depuis data/ventes.csv
```

```
[2/3] Transform (pandas)...
2 lignes invalides ecartees
```

```
[3/3] Load (PostgreSQL)...
```

Tests unitaires avec pytest

Fichier :

```
tests/test_transforms.py
```

Les tests unitaires vérifient les transformations pandas.

Les tests vérifient :

- suppression des emails vides
- suppression des montants négatifs
- conversion des types
- agrégation mensuelle
- agrégation par catégorie

Résultat obtenu

Tous les tests passent avec succès.

```
20 PASSED
```

Tests d'intégration PostgreSQL

Les tests d'intégration vérifient :

- le chargement dans PostgreSQL
- le contenu des tables
- les contraintes métier

Fichier :

```
tests/test_load.py
```

Les requêtes SQL sont utilisées pour vérifier :

- le nombre de lignes
- l'absence de montants négatifs
- l'absence d'emails vides

Couverture des tests

```
pytest --cov=src.transform --cov=src.load --cov-report=term-missing
```

Résultat obtenu

```
TOTAL 90%
```

```
ryma@DRISSRYMAHP:/mnt/c/Users/DRISS/Desktop/DRISS_Ryma_Devops/NOM_PRENOM$ pytest
--cov=src.transform --cov=src.load --cov-report=term-missing
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0
rootdir: /mnt/c/Users/DRISS/Desktop/DRISS_Ryma_Devops/NOM_PRENOM
plugins: cov-4.1.0
collected 20 items

tests/test_load.py ..... [ 25%]
tests/test_transforms.py ..... [100%]

----- coverage: platform linux, python 3.12.3-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
src/load.py                         21      4    81%    8, 60-66
src/transform.py                    20      0   100%
-----
TOTAL                               41      4    90%
```

GitHub Actions et CI/CD

Le workflow CI a été créé dans :

```
.github/workflows/ci.yml
```

Tests pytest

```
pytest -v
```

```

ryma@DRISSRYMAHP:/mnt/c/Users/DRISS/Desktop/DRISS_Ryma_Devops/NOM_PRENOM$ pytest -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /mnt/c/Users/DRISS/Desktop/DRISS_Ryma_Devops/NOM_PRENOM
plugins: cov-4.1.0
collected 20 items

tests/test_load.py::TestLoadVentes::test_ventes_propres_count PASSED [ 5%]
tests/test_load.py::TestLoadVentes::test_ventes_pas_de_montant_negatif PASSED [ 10%]
tests/test_load.py::TestLoadVentes::test_ventes_pas_email_vide PASSED [ 15%]
tests/test_load.py::TestLoadMart::test_ca_par_mois_count PASSED [ 20%]
tests/test_load.py::TestLoadMart::test_ca_par_mois_ca_positif PASSED [ 25%]
tests/test_transforms.py::TestClean::test_clean_retire_email_vide PASSED [ 30%]
tests/test_transforms.py::TestClean::test_clean_retire_montant_negatif PASSED [ 35%]
tests/test_transforms.py::TestClean::test_clean_nombre_lignes PASSED [ 40%]
tests/test_transforms.py::TestClean::test_clean_email_lowercase PASSED [ 45%]
tests/test_transforms.py::TestClean::test_clean_montant_float PASSED [ 50%]
tests/test_transforms.py::TestClean::test_clean_date_datetime PASSED [ 55%]
tests/test_transforms.py::TestClean::test_clean_ne_modifie_pas_original PASSED [ 60%]
tests/test_transforms.py::TestAggregateByMonth::test_agg_nombre_mois PASSED [ 65%]
tests/test_transforms.py::TestAggregateByMonth::test_agg_ca_total PASSED [ 70%]
tests/test_transforms.py::TestAggregateByMonth::test_agg_colonnes_presentes PASSED [ 75%]
tests/test_transforms.py::TestAggregateByMonth::test_agg_ca_positifs PASSED [ 80%]
tests/test_transforms.py::TestAggregateByCategory::test_agg_category_nombre_categories PASSED [ 85%]
tests/test_transforms.py::TestAggregateByCategory::test_agg_category_ca_total PASSED [ 90%]
tests/test_transforms.py::TestAggregateByCategory::test_agg_category_colonnes_presentes PASSED [ 95%]
tests/test_transforms.py::TestAggregateByCategory::test_agg_category_ca_positifs PASSED [100%]

```

Le workflow automatisé :

- installation des dépendances
- lancement PostgreSQL
- tests unitaires
- tests d'intégration
- couverture des tests

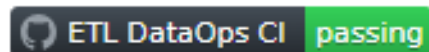
Résultat obtenu

Le workflow GitHub Actions s'exécute avec succès.

2 workflow runs		Event ▾	Status ▾	Branch ▾	Actor ▾
✓ Add bonus coverage and category aggregation ETL DataOps CI #4: Commit a02f602 pushed by gethubryma	main		🕒 2 minutes ago 🕒 51s	...	
✓ Add CI badge ETL DataOps CI #3: Commit cb18dde pushed by gethubryma	main		🕒 36 minutes ago 🕒 46s	...	
✓ Fix Python path for CI ETL DataOps CI #2: Commit 58c479d pushed by gethubryma	main		🕒 Today at 7:59 PM 🕒 47s	...	

test succeeded 13 minutes ago in 43s			Search logs	
>	✓ Set up job			1s
>	✓ Initialize containers			21s
>	✓ Check out code			2s
>	✓ Setup Python 3.11			0s
>	✓ Install dependencies			11s
>	✓ Tests unitaires pandas (sans base de donnees)			2s
>	✓ Tests integration (necessite PostgreSQL)			1s
>	✓ Post Setup Python 3.11			1s
>	✓ Post Check out code			1s
>	✓ Stop containers			0s
>	✓ Complete job			0s

Le badge CI a été ajouté dans le README.



Conclusion

Ce TP a permis de mettre en œuvre un pipeline ETL DataOps complet utilisant pandas, PostgreSQL, Docker, pytest et GitHub Actions.

Les objectifs suivants ont été atteints :

- extraction des données CSV
- nettoyage pandas
- agrégation analytique
- chargement PostgreSQL
- tests unitaires
- tests d'intégration
- automatisation CI/CD
- couverture de tests 90%

Le pipeline ETL développé fonctionne correctement et toutes les étapes ont été validées par les tests.